

Module 4

1. What is an Optimization Problem?

An optimization problem involves **finding the best solution from all feasible solutions**. In simple terms, it's about **making the best decision given certain constraints**.

1.1 Key Components of an Optimization Problem:

- **Variables (x)**: These are the **unknowns or decisions that need to be determined**. For example, if you're trying to decide how many tasks to offload from a mobile device to an edge server, the number of tasks could be your variable.
- **Domain (D)**: The domain defines **the possible values each variable can take**. For instance, if you're deciding how many tasks to offload, the domain might be any integer between 0 and 10 (i.e., you can offload anywhere from 0 to 10 tasks).
- **Constraints (C)**: These are **rules or limitations that must be satisfied**. For example, if your mobile device has limited battery power, you might have a constraint that says the total energy used for computation and communication cannot exceed a certain threshold.
- **Objective Function (f)**: This is **what you want to optimize**—either **maximize or minimize**. For example, you might want to minimize the total time taken to complete all tasks or minimize the energy consumption.

An optimization problem is defined as follows:

- Find values $\mathbf{v}_1, \dots, \mathbf{v}_n$ for the variables such that:
 1. $\mathbf{v}_i \in \mathbf{D}_i$ for each $i = 1, \dots, n$.
 2. For any constraint $\mathbf{C}_j$ relating to variables $\mathbf{x}_{j1}, \dots, \mathbf{x}_{jk}$, it holds that $(\mathbf{v}_{j1}, \dots, \mathbf{v}_{jk}) \in \mathbf{R}_j$.
 3. $\mathbf{f}(\mathbf{v}_1, \dots, \mathbf{v}_n)$ is maximum (for maximization problems) or minimum (for minimization problems) among all tuples that satisfy (1) and (2).

1.2 Simple Analogy:

Imagine you own a small bakery, and you want to decide **how many cakes (x) and pastries (y) to bake each day**.

Variables: x (number of cakes), y (number of pastries)

Domain: $x \in \{0, 1, 2, \dots, 10\}$, $y \in \{0, 1, 2, \dots, 15\}$ (you can bake up to 10 cakes and 15 pastries)

Constraints:

- You only have **enough ingredients to make 20 items** in total: $\mathbf{x} + \mathbf{y} \leq 20$
- Each **cake takes 1 hour to bake**, and **each pastry takes 0.5 hours**. You only have **12 hours available**: $\mathbf{x} + 0.5\mathbf{y} \leq 12$

Objective Function: Maximize profit.

Let's say each cake brings \$5 profit and each pastry brings \$3 profit. So, $f(x, y) = 5x + 3y$.

Your goal is to find the values of x and y that satisfy the constraints and maximize the profit.

1.3 Types of Optimization Problems

- **Maximization Problems:** Here, the goal is to **maximize the objective function**. For example, maximizing profit in the bakery scenario.
- **Minimization Problems:** Here, the **goal is to minimize the objective function**. For example, minimizing the energy consumption in a fog computing system.

Simple Example of Minimization:

Suppose you're managing a network of IoT devices and want to minimize the total energy consumption.

Variable: x (number of tasks offloaded to the edge server)

Domain: $x \in \{0, 1, 2, \dots, 10\}$

Constraint: The total latency (time delay) should not exceed 5 seconds: $\text{Latency}(x) \leq 5$

Objective Function: Minimize energy consumption: $f(x) = \text{Energy}(x)$

Here, you want to find the **optimal number of tasks to offload** such that the energy consumption is minimized while keeping the latency below 5 seconds.

2. Multi-Objective Optimization

In real-world scenarios, you often have **more than one objective to optimize, and these objectives might conflict with each other**.

For example, in fog computing, you might want to minimize both latency and energy consumption, but reducing latency might require using more energy.

2.1 Common Approaches to Handle Multiple Objectives:

- **Adding Lower Bounds:** You can **set minimum acceptable levels for some objectives and then optimize the remaining one**. For example, you might require that latency should not exceed 5 seconds ($\text{Latency} \leq 5$) and then try to minimize energy consumption.
- **Scalarizing Objectives: Combine multiple objectives into a single objective function.** For example, you could create a weighted sum of latency and energy consumption: $f(x) = w_1 * \text{Latency}(x) + w_2 * \text{Energy}(x)$, where w_1 and w_2 are weights that reflect the importance of each objective.
- **Pareto-Optimal Solutions:** Instead of finding a single solution, you **find a set of solutions where no solution is better than another in all objectives**.

For example, one solution might have low latency but high energy consumption, while another might have higher latency but lower energy consumption. Both are considered Pareto-optimal because **improving one would worsen the other**.

2.2 Simple Example of Multi-Objective Optimization:

Let's say you're designing a fog computing system where you want to minimize both the execution time and energy consumption.

Objectives: Minimize Execution Time (f_1) and Minimize Energy Consumption (f_2)

Approach: You could scalarize the objectives by creating a combined objective function: $f_{\text{combined}} = w_1 * f_1 + w_2 * f_2$, where w_1 and w_2 are weights.

Alternatively, you could look for Pareto-optimal solutions where no solution is better in both execution time and energy consumption.

3. Formalizing Practical Problems

Formalizing a practical problem into a well-defined optimization problem is a **non-trivial process**. It involves:

- **Defining Variables:** Identifying what decisions need to be made.
- **Setting Domains:** Determining the possible values for each decision.
- **Identifying Constraints:** Listing all the limitations or requirements.
- **Specifying the Objective Function:** Deciding what needs to be optimized.

3.1 Simple Example:

Consider a fog computing scenario where you want to decide how much data to process locally on a mobile device versus offloading to an edge server.

Variables: x (amount of data processed locally), y (amount of data offloaded to the edge server)

Domain: $x \in [0, 100]$, $y \in [0, 100]$ (total data is 100 MB)

Constraints:

- **Total data:** $x + y = 100$
- **Battery life:** Local processing consumes more energy, so $\text{Energy}(x) \leq 50\% \text{ of battery}$

Objective Function:

Minimize **total latency:** $f(x, y) = \text{Latency_local}(x) + \text{Latency_offload}(y)$

The goal is to find the optimal split between local processing and offloading that minimizes latency while respecting the battery constraint.

3.2 Why Formalization Matters

Formalizing the problem correctly is crucial because:

- **Impact on Complexity:** Different formalizations can lead to problems with varying computational complexity. Some formulations might be easier to solve than others.

- **Trade-offs:** You need to balance between generality (how widely applicable the problem is) and simplicity (how easy it is to solve). A very general problem might be too complex to solve efficiently, while a very simple problem might not capture all the real-world details.

3.3 Simple Example:

- Imagine you're optimizing a fog computing system for video streaming. If you formalize the problem too simply (e.g., just minimizing latency), you might end up with a solution that uses too much energy.
- On the other hand, if you include every possible detail (e.g., network conditions, user preferences, hardware specifics), the problem might become too complex to solve efficiently.
- The challenge is to find the right level of abstraction that captures the essential aspects without becoming overly complicated.

4. Why Optimization is Essential in Fog Computing

- Fog computing was developed to address several challenges in cloud computing, mobile computing, and IoT by bringing computational resources closer to end devices.
- The primary goal of fog computing is to improve certain quality attributes, such as latency, energy consumption, and compute capabilities.
- These improvements are achieved through careful optimization.

4.1 Key Drivers for Optimization in Fog Computing:

- **Minimizing Latency:** One of the main reasons fog computing exists is to reduce the time it takes for data to travel between devices and the cloud. By processing data closer to the source (on edge devices or edge servers), the system can respond faster, which is critical for real-time applications like autonomous vehicles, video streaming, or augmented reality.
- **Reducing Energy Consumption:** Many end devices, such as smartphones or IoT sensors, have limited battery life. Offloading compute-intensive tasks to nearby edge resources can save energy on these devices, extending their operational lifespan.
- **Enhancing Compute Capabilities:** End devices often have limited processing power. By offloading tasks to more powerful edge resources, fog computing enables these devices to run more complex applications that would otherwise be impossible.

4.2 Simple Example: Minimizing Latency

Imagine you're using a smartphone app that processes live video from your camera to detect objects in real-time (e.g., a security app that identifies intruders). If the app sends the video to a distant cloud server for processing, there could be a noticeable delay (latency) before the results come back. This delay could make the app less effective, especially if you need immediate feedback.

Solution: Instead of sending the video to the cloud, the app offloads the processing to a nearby edge server. Since the edge server is geographically closer, the latency is reduced, and you get near-instantaneous feedback. This is an example of how fog computing optimizes latency.

5. Balancing Multiple Objectives

In fog computing, optimization often involves **balancing multiple objectives that may conflict with each other**. For example, **reducing latency might require using more energy**, or **minimizing energy consumption might increase latency**. Therefore, optimization techniques must carefully consider trade-offs between different goals.

5.1 Common Optimization Objectives:

- **Execution Time:** How quickly a task is completed.
- **Energy Consumption:** How much energy is used by the devices and network.
- **Cost:** The financial cost of using cloud or edge resources.
- **Reliability:** Ensuring that tasks are completed even if some resources fail.
- **Security and Privacy:** Protecting data from unauthorized access or attacks.

5.2 Simple Example: Balancing Execution Time and Energy Consumption

Suppose you're designing a fog computing system for a smart home thermostat. The thermostat collects temperature data and adjusts the heating/cooling system accordingly. You want to optimize two things:

- **Execution Time:** The thermostat should adjust the temperature quickly to maintain comfort.
- **Energy Consumption:** The thermostat should use as little energy as possible to prolong battery life.

Conflict: If you **prioritize execution time**, the thermostat might process data locally on its own processor, which is fast but consumes more energy. On the other hand, if you **prioritize energy consumption**, you might offload the data processing to a nearby edge server, which saves energy but introduces some latency.

Optimization Solution: You could **find a balance by offloading only the most computationally intensive tasks to the edge server** while **keeping simpler tasks local**. This way, you achieve a good compromise between execution time and energy consumption.

6. Formal Modeling Framework for Fog Computing

Considering the hierarchical three-layer model of fog computing.



Resources and Layers

- **Cloud (c):** Represents the highest layer in the fog computing architecture, offering virtually unlimited, high-performance, and cost- and energy-efficient resources.
- **Edge Resources (E):** A set of machines offering compute services near the network edge, such as in base stations, routers, or small, geographically distributed data

centers of telecommunication providers. Each edge resource is connected to the cloud.

- **End Devices (D):** The lowest layer containing devices like mobile phones or IoT devices. Each end device is connected to one of the edge resources.

Sets and Notations

- **Set of All Resources (R):** $R = \{c\} \cup E \cup D$, which includes the cloud, all edge resources, and all end devices.
- **Compute Capacity (a(r)):** Each resource $r \in R$ is associated with a compute capacity $a(r) \in \mathbb{R}^+$ representing the **maximum computational load it can handle**.
- **Compute Speed (s(r)):** Each resource $r \in R$ has a compute speed $s(r) \in \mathbb{R}^+$, indicating **how fast it can process instructions**.
- **Marginal Energy Consumption (w(r)):** The power consumption of resource r increases by $w(r) \in \mathbb{R}^+$ for every instruction it processes.

Links and Network Characteristics

- **Set of Links (L):** $L = \{ce: e \in E\} \cup \{ed: e \in E, d \in D\}$, where:
 - ce represents links between the cloud and edge resources.
 - ed represents links between edge resources and end devices.
- **Latency (t(l)):** Each link $l \in L$ has a latency $t(l) \in \mathbb{R}^+$, representing the time it takes for data to travel across the link.
- **Bandwidth (b(l)):** Each link $l \in L$ has a bandwidth $b(l) \in \mathbb{R}^+$, indicating the maximum data transfer rate.
- **Marginal Energy Consumption of Links (w(l)):** Transmitting one more byte of data over link l increases power consumption by $w(l) \in \mathbb{R}^+$.

c	Cloud
E	Set of edge resources
D_e	Set of end devices connected to edge resource $e \in E$
D	Set of all end devices ($D = \bigcup_{e \in E} D_e$)
R	Set of all resources ($R = \{c\} \cup E \cup D$)
$a(r)$	Compute capacity of resource $r \in R$
$s(r)$	Compute speed of resource $r \in R$
$w(r)$	Marginal energy consumption of resource $r \in R$
L	Set of all links between resources

$t(l)$	Latency of link $l \in L$
$b(l)$	Bandwidth of link $l \in L$
$w(l)$	Marginal energy consumption of link $l \in L$

6.1 Importance of These Notations

These notations and sets are crucial for formally defining optimization problems in fog computing. They allow researchers and practitioners to:

- **Model the System Accurately:** By defining the resources, their capacities, and the network characteristics, one can create precise models of fog computing systems.
- **Formulate Constraints and Objectives:** The notations help in specifying constraints (like latency and bandwidth limits) and objectives (like minimizing energy consumption or execution time).
- **Enable Comparative Analysis:** With a standardized set of notations, different optimization approaches and algorithms can be compared and combined more effectively.

6.2 Simple Example

Imagine a smart home security system that includes several IoT devices such as cameras, motion sensors, and door/window sensors. These devices are connected to an edge server located in the homeowner's local network (e.g., a router or a small server). The edge server is further connected to a cloud data center for additional processing and storage.

Objective: The goal is to **minimize the total energy consumption of the system while ensuring that video footage from the cameras is processed with minimal latency** (to detect intruders quickly).

Representations

1. Resources and Layers

- **Cloud (c):** The cloud data center where long-term storage and heavy processing (like video analysis) can occur.
- **Edge Resources (E):** A single edge server located in the home network.
- **End Devices (D):** IoT devices such as cameras, motion sensors, and door/window sensors.

2. Sets

- **Set of All Resources (R):** $R = \{c\} \cup E \cup D$, which includes the cloud, the edge server, and all IoT devices.
- **Set of Links (L):** $L = \{ce\} \cup \{ed: d \in D\}$, where:
 - ce represents the link between the cloud and the edge server.
 - ed represents the links between the edge server and each IoT device.

3. Resource Characteristics

Compute Capacity ($a(r)$):

- $a(c)$ = 1000 instructions per second (high capacity).
- $a(e)$ = 500 instructions per second (moderate capacity).

- $a(d)=50$ instructions per second for each IoT device (low capacity).

Compute Speed ($s(r)$):

- $s(c)=2$ GHz (fast processing).
- $s(e)=1.5$ GHz (moderate speed).
- $s(d)=0.5$ GHz for each IoT device (slow processing).

Marginal Energy Consumption ($w(r)$):

- $w(c)=0.1$ watts per instruction (efficient due to economies of scale).
- $w(e)=0.2$ watts per instruction.
- $w(d)=0.5$ watts per instruction for each IoT device (high energy consumption due to battery limitations).

4. Link Characteristics

Latency ($t(l)$):

- $t(ce)=50$ ms (latency between cloud and edge server).
- $t(ed)=10$ ms for each link between the edge server and IoT devices.

Bandwidth ($b(l)$):

- $b(ce)=100$ Mbps (high bandwidth between cloud and edge server).
- $b(ed)=10$ Mbps for each link between the edge server and IoT devices.

Marginal Energy Consumption of Links ($w(l)$):

- $w(ce)=0.05$ watts per byte (energy cost for transmitting data between cloud and edge server).
- $w(ed)=0.1$ watts per byte for each link between the edge server and IoT devices.

Optimization Problem Representation

Variables (x):

- $x_{local}(d)$: Amount of data processed locally on each IoT device d .
- $x_{edge}(d)$: Amount of data offloaded from each IoT device d to the edge server.
- $x_{cloud}(e)$: Amount of data offloaded from the edge server e to the cloud.

Domain (D):

- $x_{local}(d) \in [0,100]$ MB (each IoT device can process up to 100 MB locally).
- $x_{edge}(d) \in [0,100]$ MB (each IoT device can offload up to 100 MB to the edge server).
- $x_{cloud}(e) \in [0,500]$ MB (the edge server can offload up to 500 MB to the cloud).

Constraints (C):

1. **Total Data Constraint:** Each IoT device must process or offload all its data.
 - $x_{local}(d) + x_{edge}(d) = 100$ MB for each $d \in D$.
2. **Latency Constraint:** The total latency for processing video footage should not exceed 200 ms.
 - $t(ed) \times (x_{edge}(d) / b(ed)) + t(ce) \times (x_{cloud}(e) / b(ce)) \leq 200$ ms.

3. **Energy Constraint:** The total energy consumption should not exceed 10 watts.

$$w(d) \times x_{local}(d) + w(e) \times x_{edge}(d) + w(c) \times x_{cloud}(e) \leq 10 \text{ watts.}$$

Objective Function (f):

- Minimize the total energy consumption:

$$f(x) = \sum_{d \in D} [w(d) \times x_{local}(d) + w(e) \times x_{edge}(d)] + w(c) \times x_{cloud}(e)$$

7. Metrics

Metric	Sub-Metric	Description	Relevance to Fog Computing
Performance	Execution Time, Latency, and Throughput	- Performance is measured by how quickly tasks are completed. - Involves computation time on devices and data transfer times across layers (end devices, edge, cloud). - Some tasks can run in parallel, while others must be sequential.	- Critical for latency-sensitive applications like real-time video streaming or autonomous vehicles. - Total execution time depends on both computation and communication delays. - Tasks often involve multiple resources, so optimizing the critical path is essential.
	Critical Path	The longest sequence of dependent tasks determines overall task completion time	Includes both computation and communication delays, impacting total execution time.
Resource Usage	CPU and Memory	- Efficient use of limited CPU and memory is crucial, especially for end devices. - End devices have constrained capabilities, making resource optimization vital.	- Overusing CPU slows down applications, overbooking memory causes failures. - Edge resources also have limited capacities, requiring careful load balancing.
	Network Bandwidth	Bandwidth between devices, edge resources, and the cloud is often limited.	Optimizing bandwidth usage avoids bottlenecks and ensures smooth data flow.
Energy Consumption	Power vs. Energy	- Power consumption refers to the rate of energy use, while energy consumption considers total energy over time. - Idle resources still consume energy, but usage increases power consumption.	- Moving tasks to faster resources (even if they consume more power) can reduce overall energy usage. - For end devices, minimizing energy consumption extends battery life.
	Battery Life	Minimizing energy consumption is critical for battery-powered end devices.	Edge resources and cloud servers optimize energy usage for cost efficiency and environmental sustainability.
Financial Costs	Infrastructure and Network Costs	Using cloud or edge resources may incur fixed or usage-based costs.	Transferring data over networks may have associated costs.

		Financial optimization minimizes these expenses while maintaining performance and reliability.	Cost minimization is essential for economically viable fog deployments.
Further Quality Attributes	Reliability	Ensuring system reliability involves redundancy, such as having multiple resources perform the same task.	Redundancy helps handle failures but increases costs.
	Security and Privacy	- Security can be enhanced by using trusted resources and minimizing co-location of tasks from different users. - Co-location of tasks belonging to the same user reduces network-based attacks.	Privacy risks can be reduced by anonymizing data and limiting data exposure.
	Trade Offs	Improving reliability, security, or privacy often comes at the expense of higher costs, increased latency, or greater energy consumption.	Balancing these trade-offs is a key challenge in fog computing optimization.

8. Optimization Opportunities along the Fog Architecture

It explains how optimization problems in fog computing can be categorized based on which layers of the fog architecture are involved.

Fog computing typically has a three-layer structure:

- **Cloud Layer:** The highest layer, where powerful data centers provide virtually unlimited compute power.
- **Edge Layer:** The middle layer, consisting of edge resources like routers, base stations, or small data centers that are closer to end devices.
- **End Devices Layer:** The lowest layer, including IoT devices, smartphones, or sensors with limited computational power.

8.1 Why Single-Layer Optimization Isn't Enough

If only one layer (e.g., just the cloud or just the end devices) is involved, it's not really "fog computing."

For example:

- Optimizing only the cloud would be a pure cloud computing problem.
- Optimizing only end devices would fall under mobile computing or IoT.
- Real fog computing problems require collaboration between at least two layers to take advantage of the strengths of each layer.

8.2 Types of Optimization Problems Based on Layers

i) Cloud + Edge Resources [Two Layer Optimization Problem]

- This type of optimization involves both the cloud and edge resources.
- **Example:** Minimizing the overall energy consumption of the system while ensuring that tasks are completed within acceptable latency limits.
- **Why it matters:** Edge resources are closer to users, so they reduce latency compared to the cloud. However, the cloud has more compute power, so some tasks may still need to be sent there.

ii) Edge Resources + End Devices [Two Layer Optimization Problem]

- This type of optimization focuses on collaboration between edge resources and end devices.
- **Example:** Offloading compute-intensive tasks from end devices (like smartphones or IoT sensors) to nearby edge resources to save battery life on the devices.
- **Why it matters:** End devices often have limited processing power and battery, so offloading tasks to edge resources helps improve performance and extend device life.

iii) All Three Layers Together (Cloud + Edge + End Devices)

- This type of optimization considers all three layers: cloud, edge resources, and end devices.
- **Example:** Distributing tasks across all layers to balance performance, energy consumption, and costs.
- **Why it's challenging:** Optimizing across all three layers is complex because:
 - There are many variables to consider (e.g., task placement, data transfer, resource allocation).
 - Different stakeholders (e.g., cloud providers, network operators, device owners) may have conflicting goals.
 - Changes in one layer (e.g., adding new edge resources) can affect the entire system.

8.3 What Can Be Optimized?

Within each layer, optimization can focus on different aspects:

- **Data Distribution:** Deciding where to store and process data (e.g., should data stay on the device, go to the edge, or be sent to the cloud?).
- **Code Placement:** Deploying program code on the most suitable resources (e.g., running part of an app on the edge server and part on the cloud).
- **Task Splitting:** Dividing tasks among multiple resources (e.g., splitting a computation-heavy task between the device and the edge).

8.4 Distributed Optimization

- Fog computing systems are distributed by nature, meaning decisions often need to be made locally by individual resources (e.g., an edge server deciding how to handle incoming tasks).
- **Goal:** Ensure that local optimizations lead to a globally optimal system.
- **Challenge:** Coordinating decisions across multiple autonomous resources without creating bottlenecks or inefficiencies.

9. Optimization Opportunities along the Service Life Cycle

It explains how optimization in fog computing can be applied at different stages of a service's life cycle. **Fog computing**, like cloud computing, **involves the provision and consumption of services**, and each stage of the service life cycle presents unique opportunities for optimization.

9.1 Overview of the Service Life Cycle

The service life cycle in fog computing consists of three main stages:

- **Design Time:** When the **service is being designed**.
- **Deployment Time:** When the **service is being deployed** on specific resources.

- **Run Time:** When the service is operational and actively used.

Each of these stages has different optimization opportunities based on the information available and the goals of the system.

9.2 Design-Time Optimization

What Happens at Design Time?

At this stage, the fog service is being planned and designed. However, exact details about the end devices (e.g., their types, capabilities, or numbers) are typically not available. Instead, optimization focuses on the cloud and edge layers, where more information is known.

Key Optimization Opportunities:

- **Cloud and Edge Layers:** Optimize resource allocation, task distribution, and data placement between the cloud and edge resources.
- **End Devices:** Focus on general design decisions, such as the types of devices that will interact with the system (e.g., IoT sensors, smartphones). Specific device instances are unknown at this stage.

Example:

Imagine designing a fog-based video streaming service. At design time, you might decide:

- How much data should be cached at edge resources to reduce latency.
- Which types of devices (e.g., smartphones, smart TVs) the service should support.

9.3 Deployment-Time Optimization

What Happens at Deployment Time?

During deployment, the service is assigned to specific resources (e.g., edge servers, cloud data centers). More detailed information about the resources becomes available, enabling further optimization.

Key Optimization Opportunities:

- **Resource Allocation:** Use the known capacities of edge resources to optimize how tasks are split between the cloud and edge.
- **Task Placement:** Decide which tasks should run on which resources based on their availability and capabilities.
- **Data Distribution:** Determine where data should be stored and processed to minimize latency and energy consumption.

Example:

Continuing with the video streaming example:

- You might allocate specific edge servers to handle requests from nearby users.
- You could pre-load popular videos onto edge caches based on regional demand patterns.

9.4 Run-Time Optimization

What Happens at Run Time?

Once the service is operational, real-time information about the system becomes available. This includes details about the end devices (e.g., their compute capacity, battery status)

and the tasks they want to offload. Since these factors change dynamically, continuous optimization is required.

Key Optimization Opportunities:

- **Dynamic Task Offloading:** Decide in real time which tasks should be offloaded to edge resources or the cloud based on current conditions (e.g., device battery levels, network congestion).
- **Load Balancing:** Balance computational load across edge resources to prevent bottlenecks.
- **Adaptation to Changes:** Continuously monitor system parameters (e.g., latency, energy consumption) and re-optimize as needed.

Challenges:

- **Time Constraints:** Optimization algorithms must be fast because decisions need to be made in real time.
- **Cost of Changes:** Reconfiguring the system (e.g., moving tasks between resources) may incur costs, so these must be considered during optimization.

Example:

In the video streaming service:

- If a sudden spike in user demand occurs, the system might dynamically redirect some requests to the cloud to handle the increased load.
- If an edge server fails, the system might redistribute its tasks to other nearby edge resources.

10. Toward a Taxonomy of Optimization Problems in Fog Computing

It aims to provide a structured framework for classifying and understanding the various optimization problems that arise in fog computing.

10.1 Purpose of the Taxonomy

The goal of creating a taxonomy is to systematically classify optimization problems in fog computing. By organizing these problems into categories, researchers and practitioners can:

- Compare different approaches more easily.
- Identify gaps in existing research.
- Understand the relationships between various optimization problems.

This taxonomy helps highlight the diversity of optimization challenges in fog computing and provides a standard reference for future work.

10.2 Classification Dimensions

The taxonomy is built around several key dimensions that define optimization problems in fog computing. These dimensions include:

a. Context / Domain

This refers to the specific application or scenario being optimized.

Examples include:

- Video streaming services: Optimizing data rates for video delivery across edge resources.
- Computation offloading: Deciding which tasks to offload from end devices to edge resources.
- Robot collaboration: Distributing compute tasks among cooperating robots connected to an edge server.

b. Considered Metrics

Each optimization problem focuses on specific metrics that need to be optimized or constrained. Common metrics include:

- **Performance**: Execution time, latency, throughput.
- **Resource Usage**: CPU, memory, network bandwidth.
- **Energy Consumption**: Total energy used by devices, edge resources, and the cloud.
- **Financial Costs**: Infrastructure and network costs.
- **Quality Attributes**: Reliability, security, privacy.

For example:

- In video streaming, the focus might be on maximizing utility (e.g., weighted data rate) while minimizing energy consumption in the cloud.
- In computation offloading, the focus might be on minimizing energy consumption of end devices and the total time to transfer and execute tasks.

c. Considered Layers / Resources

Optimization problems are classified based on which layers of the fog architecture are involved:

- **Cloud + Edge Resources**: Problems involving resource allocation and task distribution between the cloud and edge resources.
- **Edge Resources + End Devices**: Problems focusing on offloading tasks from end devices to edge resources.
- **All Three Layers (Cloud + Edge + End Devices)**: Problems that optimize across all layers, though these are less common due to their complexity.

d. Phase in the Service Life Cycle

Optimization opportunities are also categorized based on when they occur in the service life cycle:

- **Design Time**: Focuses on planning and designing the system without exact details about end devices.
- **Deployment Time**: Uses available information about resources to optimize task placement and resource allocation.
- **Run Time**: Dynamically optimizes the system during operation based on real-time data about devices, tasks, and network conditions.

e. Optimization Algorithm

Different optimization techniques are used depending on the problem's complexity and constraints:

- **Exact Algorithms:** Provide optimal solutions but may be computationally expensive.
- **Heuristics:** Provide approximate solutions quickly, often used for complex or nonconvex problems.
- **Distributed Algorithms:** Account for the distributed nature of fog computing systems, where decisions are made locally by autonomous resources.

10.3 Examples of Classified Optimization Problems

Example 1: Video Streaming Service (Do et al. [21])

- **Context / Domain:** A video streaming service with a central cloud serving distributed edge resources, which in turn serve end devices.
- **Considered Metrics:**
 - Utility (weighted data rate of edge resources).
 - Compute capacity of the cloud data center.
 - Energy consumption of the cloud data center.
- **Considered Layers / Resources:** Cloud and edge resources.
- **Phase in Life Cycle:** Design / deployment time.
- **Optimization Algorithm:** Distributed iterative improvement inspired by the ADMM method.

Example 2: Computation Offloading (Sardellitti et al. [22])

- **Context / Domain :** Mobile edge computing (MEC) where mobile devices offload compute tasks to a nearby edge resource.
- **Considered Metrics:**
 - Energy consumption of end devices.
 - Total time to transfer and execute offloaded tasks.
 - Amount of compute power of the edge resource occupied by offloaded tasks.
- **Considered Layers / Resources:** Edge resources and end devices.
- **Phase in Life Cycle:** Run time.
- **Optimization Algorithm:** Iterative heuristic using successive convex approximation.

Example 3: Robot Collaboration (Mushunuri et al. [23])

- **Context / Domain:** Cooperating mobile robots sharing compute tasks with an edge server.
- **Considered Metrics:**
 - Communication cost between end devices and the edge resource.
 - Battery power of end devices.
 - CPU capacity of end devices and the edge resource.
- **Considered Layers / Resources:** Edge resources and end devices.
- **Phase in Life Cycle:** Run time.
- **Optimization Algorithm:** Nonlinear optimization using the COBYLA algorithm within the NLOpt library.